

## Stage - 1 : Decide on a Problem

⇒ Sometime we don't exactly know what the problem is  
↳ Not specific enough problem

⇒ So you start breaking it down —  
If the model doesn't give you the desired / Accurate result formulation  
↳ Investigate and refine the problem

- You look at data available at time
- What data you have access to
- What data is cheap or expensive
- Or how much data you have
- Or the Architecture.

(Trial and error involved)

⇒ You would design a model like you would design an iPhone or product.  
↳ Problem → Formulation → Experiment → Evaluate —  
→ Result → Refine → Repeat → Final Solution

- If you are modeling some aspect of physical system you are kind of inherently doing this intersection of Physics and ML.
- ML Model should be capturing some physics. But we wanna go in depth.

⇒ Data Science In general is process of Asking and Answering Questions from data centric perspective.

People look at ML as a solution to a problem, they be building ML models just because they can

[Computers are useless, they can only give you answers. — Picasso]

↳ Data driven models can only model your data.

↳ You have to be the one to decide what is the right model, what is the right fidelity, what would be a useful model and how are you gonna use it down the stream.

That's a human endeavor —

Computers / ML can perform the  
imitation but human brain

That's a human endeavor

- Why do you need ML model?

- ↳ You better understand why you need this ML model in first place.

- ↳ And how are you gonna use this.

Computers / ML can perform the computation, but human have to formulate the question and decide what constitutes a useful Answer.

i) Discovering / Learning new Physics: We have types of data, we can start using that data to build and improve on our first principles physics models to learn new physics that we've never been able to write down before. Then we use those models like physics based models. → Then apply this to a much more complicated system where humans haven't been able to discover that physics like equals.

Now the answer to the question:—

ii) Capture Expensive Physics:

To discover / learn new physics when existing physics based models aren't enough.

- "Surrogate" modeling
- Faster while Retaining Accuracy
- Automatic Differentiation for optimization.

⇒ Surrogate: a fast, simplified mathematical or ml approximation that stands in for complex, slow or expensive system.

- ML models are often inherently automatically differentiable
- Meaning you can compute the sensitivity of outputs to inputs using back propagation, things that we use to train NN and ML models.
- You can use that automatic differentiability of that model in classical engineering design and optimization loops. ⇒ shape optimization of a wing.

Many models are differentiable, and automatic differentiation let us efficiently compute their derivatives

→ Capturing Multiphysics Interactions Example: Simulating Motions of Planets

## → Capturing Multiphysics Interactions

### ↳ Discrepancy Modeling

Example: Cloud Physics → Way that cloud forms  
Ice Crystals → Precipitation → Condensation  
Multiscale Multiphysics Process

Example: Simulating Motions of Planets  
⇒ that have tidal forces and oceans and multiphysics interactions that becomes really complicated quickly

Read about this! ⇒ Update them with new data: Updating system that may change with time

## Super Resolution tasks

→ Ideas from image sciences

→ Super important for Physics applications.

→ You have a low resolution image and statistical info you collected over lots of lots of data you might be able to fill in that low resolution image into a high resolution image.

## Discover new Physics

→ Can we discover new physics from observational data?

→ It is a Problem statement in itself

→ What if there is something in our data that we can explain

→ Can we start discovering new physics to explain discrepancies

Note

Discrepancy Modeling: Use ML model to model the difference between an existing physics based model and observations / reality

Reality = Physics model + Discrepancy

## Materials Discovery

## Conceptual (Bio) Chemistry

Faster drug discovery

Faster simulation of

Molecules

→ You want your ML model to respect Physics

→ Faster protein design

→ Digital Representation of

Molecules

## Digital twin and discrepancy Model

→ Digital Representation of physical assets and how it behaves.

→ How a physical model behaves over time and use that data for a digital twin of that object for simulation.

Example: A Robotic Arm

- ↳ have non linear bearing chatter
- ↳ theres wind resistance
- ↳ things that would cause small discrepancies

Between actual device and your idealized physics model.

- So we might be able to model that discrepancy and close the loop and make our digital twin more accurate using ML model.

Digital twin = Physics Model + ML discrepancy Model

## Shape optimization

→ Building Artificial hearts

→ Designing Wing for an Aircraft

- ↳ Optimizing the shape of an object that's going through fluid → liquid or gases

- ↳ Maximize lift, minimize drag or maximize down force.
- ↳ Substance that can flow and continuously deform when a shear force is applied

## Multiobjective optimization

→ What materials

- ↳ Maintenance gonna be easy or hard

→ What would be the weight

→ How reliable is it

Multiobjective Optimization Problems

# We can build a model that spits out lots of figure of merits and quantification of interests and we can use that Machine learning model as surrogate to do this kind of complex multiobjective optimization over.

# If objective function changes you can tweak your objective function and you can still optimize that ML surrogate model.

In simple words: Instead of having an expensive physics simulation we can build

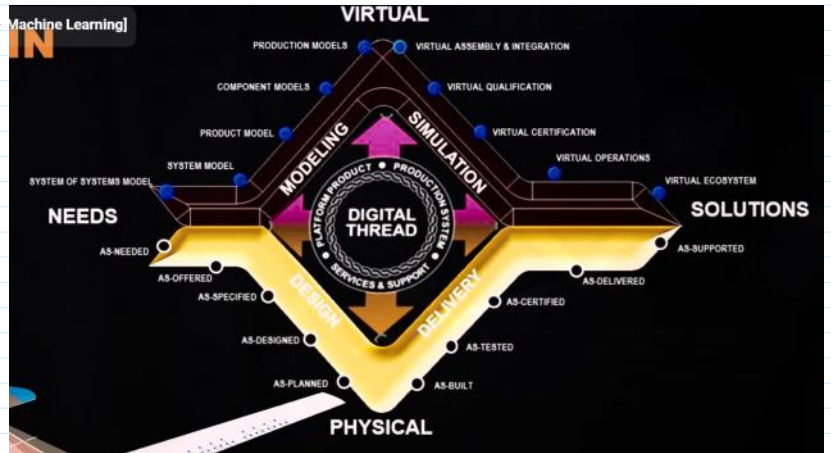
you can still optimize that ML surrogate model.

In simple words: Instead of having an expensive physics simulation repeatedly calculate everything, you could train an ML surrogate that predicts several quantities:

Design  $\rightarrow$  ML Surrogate  $\rightarrow$   $\left\{ \begin{array}{l} \text{Lift} \\ \text{Drag} \\ \text{Reliability} \\ \dots \end{array} \right.$

• This is exactly the idea of digital twin:

- Hierarchy of multi fidelity models
- Hybrid Physics-based and ML/AI
- End to End adjoint free optimization.



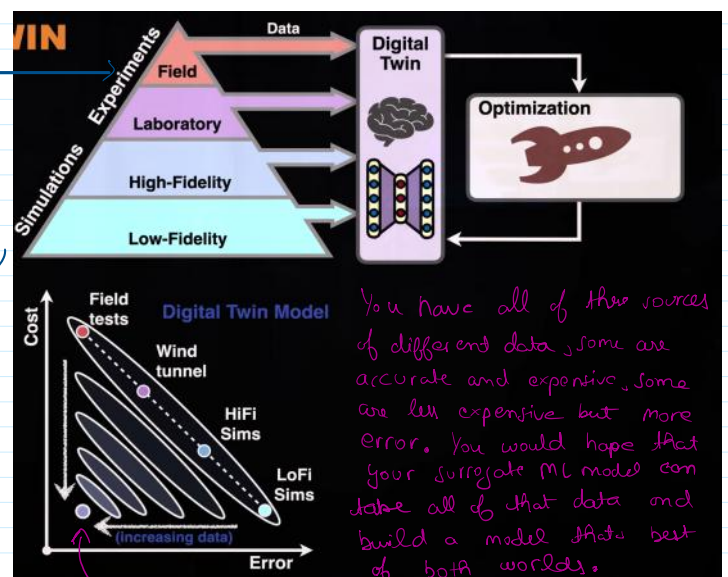
$\Rightarrow$  If we have a ML model like NN to describe some piece of this digital twin, the same kind of Automatic differentiability of that NN that we used to train the model, to back propagate the loss function signals to tweak the parameter of that model

$\hookrightarrow$  We can also use that in the actual optimization process, eg.: if we are trying to optimize that aircraft or a wind turbine

$\hookrightarrow$  If our model of that is automatically differentiable like most ML models are, we might be able to this kind of adjoint free optimization, easier optimization, cheaper.

$\Rightarrow$  Goal is to build a surrogate model that takes in that data and predict how that system will behave in different use cases, different scenarios.

$\Rightarrow$  Then if the surrogate model is good enough we might be able to do lot of optimization in virtual world cutting the costs in real world

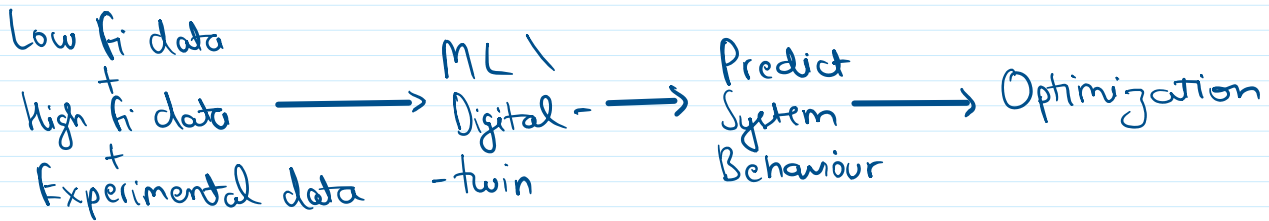


You have all of those sources of different data, some are accurate and expensive, some are less expensive but more error. You would hope that your surrogate ML model can take all of that data and build a model that's best of both worlds.

no lot of optimization in virtual world cutting the costs in real world and building physical models.



Lower cost model  
Lower Error



Dynamics

→ Modeling a system as it evolves forward in time might be useful but because the system is chaotic if we have small uncertainty in initial condition its gonna massively amplify in the future.

↳ So getting a deterministic prediction of the behaviour of the system forward in time might be too much to ask for.

↳ May be what we want is a probability of being at some location in a future time given its initial condition.

Eg: We want to know what will be weather 5 days from now, that is one trajectory and we have to get that one trajectory right.

↳ But if we wanna know climate 50 years from now, we are looking at probabilities on distributions that goes through that dynamical system.

=>

$$x_0 \rightarrow x(t)$$

One initial condition → One predicted trajectory

=>

$$x_0 \rightarrow p(x(t))$$

Initial condition + uncertainty → distribution of possible future states

Initial condition + uncertainty → Distribution of possible future states

=> For chaotic dynamical systems, uncertainty can grow with time, making probabilistic predictions more meaningful than a single deterministic trajectory.

Chaos => Changes the kind of problems you can and cannot answer - formulated.

↳ If you know your system is chaotic that should change the way you set up your problem statement.

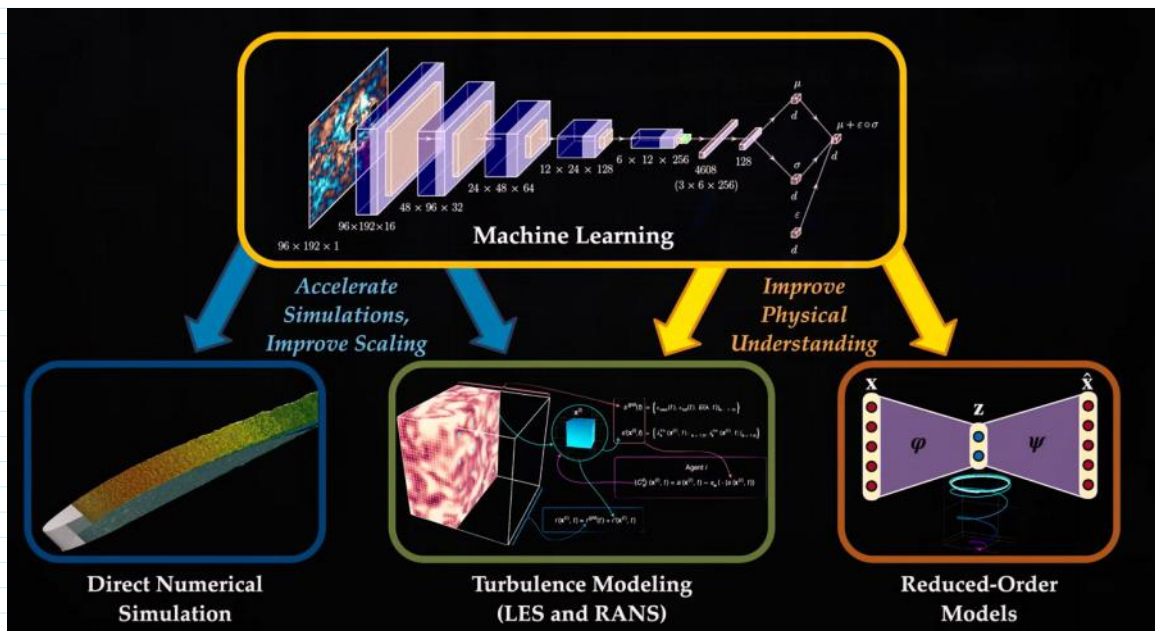
Example: Climate modeling [Nature of Physical system → Type of ML model]

Example: Urmare modeling

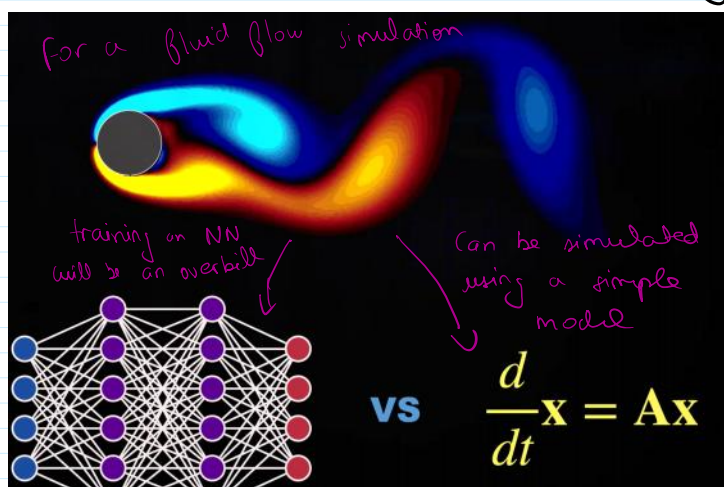
## Dynamic Systems and Control Benchmarks :

- ↳ We need bench marks systems for dynamical system, engineering systems, for control systems, so we can test our ML models and make sure that they are actually solving the tasks we want them to be.
- ↳ And to learn which task is good for which model (like for fluid, for mechanical systems, etc)
- Static data (simulated or Experiment)
- Interactive data (simulated only)
- Living and physical (Cyberphysical experiment)

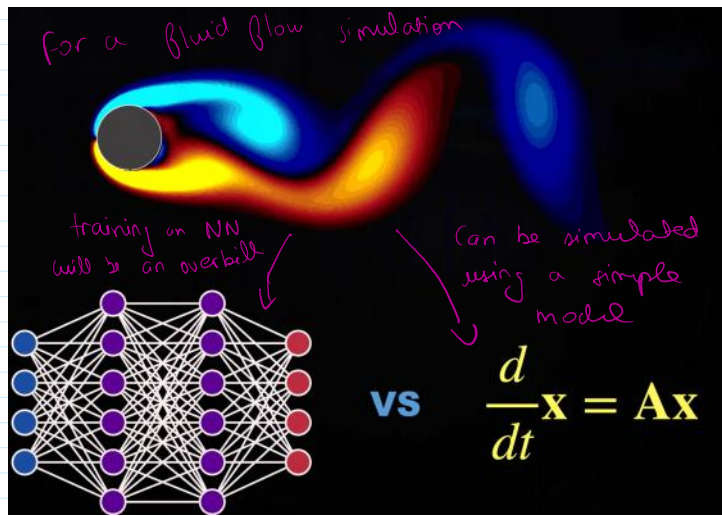
## ML for Computational Fluid Dynamics :



## When not to use Machine Learning :



- ↳ Ask question → do i need ML for this problem
  - ↳ Is my physical based model good enough?
  - ↳ Is there a simpler method?
- If something simple works, then use that.



- ↳ Ask question → do i need ML for this problem
  - ↳ Is my physics based model good enough?
  - ↳ Is there a simpler method?
- If something simple works, then use that.